SOUM DECO — Engineering Plan for 1 Million Visits/Month

Traffic Profile Analysis

Before designing solutions, we must understand what 1 million monthly visits (mostly new visitors) actually means in terms of infrastructure load:

Metric	Value
Daily visits (average)	33,333
Peak hour visits (4x average)	5,555/hour
Peak requests/second	~1.5 RPS (page loads)
Total HTTP requests/month	~15 million (15 requests per visit)
Product data API calls/month	~2 million
Image requests/month	~4 million
Estimated bandwidth/month	1.5–2 TB
Orders/month (2% conversion)	~20,000

The key challenge with "mostly new visitors" is that **browser caching provides almost no benefit** — every visitor needs a fresh data fetch, fresh images, and a fresh JavaScript bundle. This makes server-side caching and CDN strategy absolutely critical.

Current Architecture Failure Points at 1M Visits

Component	Current Limit	Required Capacity	Gap
Google Apps Script	20,000 calls/day (free)	66,666 calls/day	**3.3x over limit**
Apps Script execution	90 min/day	~500 min/day	**5.5x over limit**
Cloudinary bandwidth	25 GB/month	100–600 GB/month	**4–24x over limit**
KV reads (free)	100,000/day	66,666/day (if used)	Borderline OK

Cloudflare Pages bandwidth	Unlimited	Unlimited	OK
Cloudflare Pages files	20,000 (free)	254 files	OK

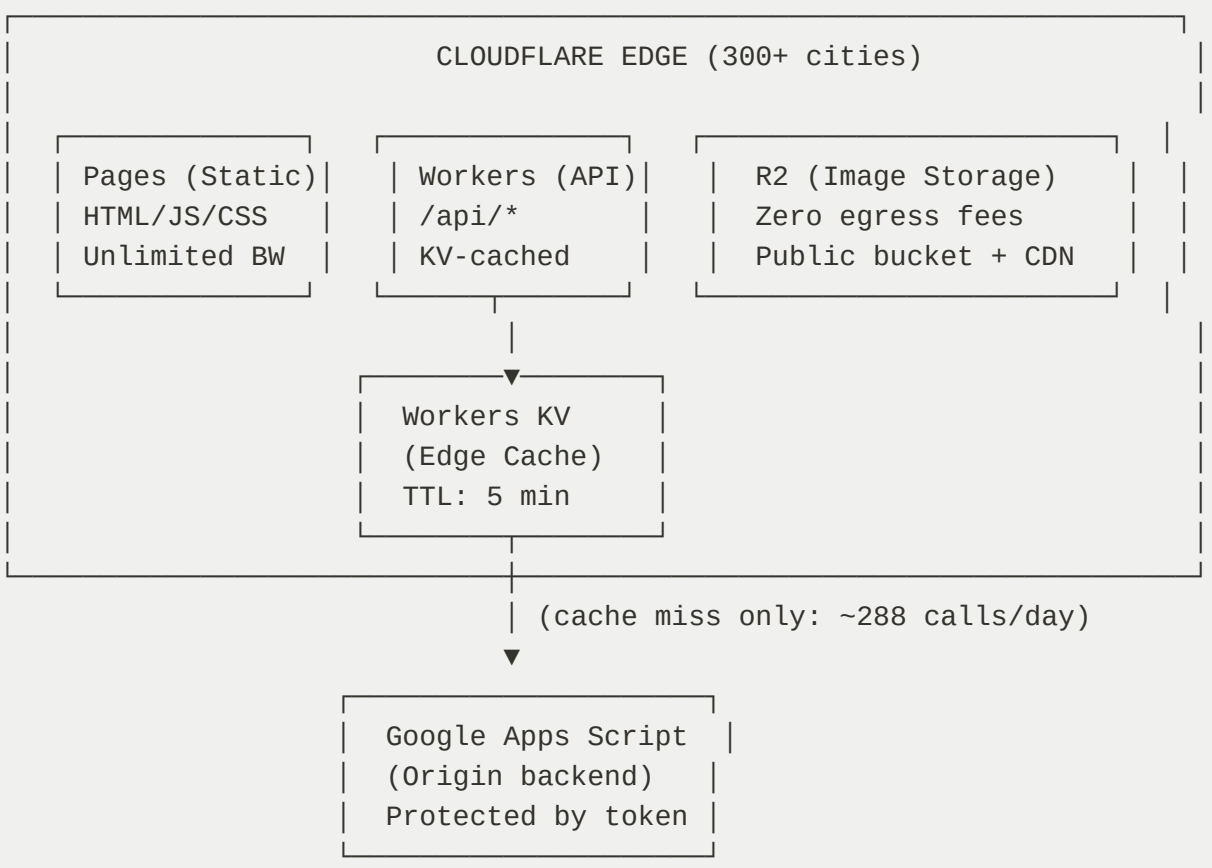
Verdict: The current architecture would catastrophically fail at ~150K visits/month, well before reaching 1M.

Solution Architecture (Target: 1M+ Visits/Month)

The redesigned architecture eliminates all single points of failure and leverages Cloudflare's edge network for near-zero-latency responses to every visitor.

Architecture Diagram

Plain Text



Key Principle: Eliminate Direct Client-to-Origin Calls

Currently, every visitor's browser calls Google Apps Script directly. At 1M visits/month, that's 2M+ direct calls to a service limited to 20K/day.

The solution: Interpose a Cloudflare Workers edge cache (KV) between visitors and Apps Script. With a 5-minute cache TTL, 33,333 daily visitors become only **288 origin calls/day** (1440 minutes ÷ 5 minutes = 288 cache refreshes).

Phase 1: Critical Infrastructure (Week 1)

1.1 — Migrate from @cloudflare/next-on-pages to @opennextjs/cloudflare

Why: The current `@cloudflare/next-on-pages` adapter is incompatible with Next.js 16, causing all API routes to return 500 errors. The `@opennextjs/cloudflare` adapter fully supports Next.js 16 (all minor and patch versions), including SSR, ISR, API routes, and middleware.

Action:

Bash

```
# Remove old adapter
npm uninstall @cloudflare/next-on-pages

# Install OpenNext adapter
npm install -D @opennextjs/cloudflare

# Update build script in package.json
# "build:cloudflare": "npx @opennextjs/cloudflare"
```

Update `wrangler.toml`:

Plain Text

```
name = "soumdeco"
compatibility_date = "2025-04-01"
compatibility_flags = ["nodejs_compat"]
main = ".open-next/worker.js"
assets = { directory = ".open-next/assets", binding = "ASSETS" }

[[kv_namespaces]]
binding = "CATALOG_KV"
id = "ec54ba6bef24403cb9082e6472fb851b"
```

Impact: Restores all 6 API routes to working condition. Enables server-side KV caching. Enables ISR (Incremental Static Regeneration) for product pages.

Cost: $0 (same free tier).

1.2 — Implement Edge KV Caching for Product Data

Why: This single change reduces Apps Script calls from 2,000,000/month to ~8,640/month (a 99.6% reduction).

Modify `/api/products/route.ts` **:**

TypeScript

```
export const runtime = "edge";

export async function GET(req: NextRequest) {
  const env = (req as any).env || (globalThis as any).env;

  // 1. Try KV cache first (edge-fast, <5ms)
  if (env?.CATALOG_KV) {
    const cached = await env.CATALOG_KV.get("products_v2", "json");
    if (cached) {
      return NextResponse.json(cached, {
        headers: {
          "Cache-Control": "public, max-age=60, stale-while-revalidate=240",
          "X-Cache": "HIT",
        },
      });
    }
  }

  // 2. Cache miss — fetch from Apps Script (happens every 5 min)
  const sheetUrl = getSheetBaseUrl();
  const res = await fetch(`${sheetUrl}?action=products`, {
    headers: { "X-Auth-Token": process.env.APPS_SCRIPT_SECRET },
  });
  const data = await res.json();

  // 3. Store in KV with 5-minute TTL
  if (env?.CATALOG_KV) {
    await env.CATALOG_KV.put("products_v2", JSON.stringify(data), {
      expirationTtl: 300, // 5 minutes
    });
  }

  return NextResponse.json(data, {
```

```
    headers: {
      "Cache-Control": "public, max-age=60, stale-while-revalidate=240",
      "X-Cache": "MISS",
    },
  });
}
```

Update the client to fetch from the edge API (not Apps Script directly):

TypeScript

```
// In use-catalog.ts — change clientListProducts to use edge API
const url = "/api/products"; // Edge-cached, <50ms globally
```

Impact:

- Apps Script calls: 2,000,000/month → 8,640/month
- Response time: 2-5 seconds (Apps Script) → <50ms (KV edge)
- Eliminates Apps Script rate limit risk entirely

Cost: Workers Paid plan ($5/month) for unlimited KV reads.

1.3 — Migrate Images to Cloudflare R2 with Public Bucket

Why: Cloudinary's free tier (25 GB bandwidth) would be exhausted in 1-2 days at 1M visits/month. Cloudflare R2 has **zero egress fees** — serving images is essentially free regardless of traffic.

Action:

1. Create R2 bucket and enable public access:

Bash

```
npx wrangler r2 bucket create soumdeco-images
# Enable public access via custom domain: images.soumdeco.com
```

1. Upload all 109 existing images (currently 2.6 MB total — trivial):

Bash

```
npx wrangler r2 object put soumdeco-images/products/ --file=public/images/produ
```

1. Update image URLs to use R2 public domain:

TypeScript

```
// In image-manifest.ts or use-catalog.ts
const IMAGE_BASE = "https://images.soumdeco.com/products";
```

1. Update the auto-sync script to upload to R2 instead of committing to Git.

Cost calculation for 1M visits/month:

Item	Usage	Free Tier	Overage	Cost
Storage	~500 MB (higher-res images)	10 GB free	$0	**$0**
Class B reads	4M image requests	10M free	$0	**$0**
Egress	100-600 GB	Free (always)	$0	**$0**
Total				**$0/month**

R2 is essentially free for this use case because the free tier covers 10M reads/month and egress is always free.

1.4 — Add HTTP Cache Headers for Static Assets

Why: For the ~20% of visitors who browse multiple pages (or return within a session), proper cache headers prevent redundant downloads.

Update `public/_headers`:

Plain Text

```
# All static assets — immutable (filename-hashed by Next.js)
/_next/static/*
  Cache-Control: public, max-age=31536000, immutable

# Product images from R2
/images/products/*
  Cache-Control: public, max-age=86400, stale-while-revalidate=604800

# HTML pages — short cache + revalidation
/
  Cache-Control: public, max-age=0, s-maxage=300, stale-while-revalidate=600

# API responses — edge-cached
/api/products
```

```
  Cache-Control: public, max-age=60, s-maxage=300, stale-while-revalidate=600

/api/stock
  Cache-Control: public, max-age=60, s-maxage=180, stale-while-revalidate=300
```

Impact: Reduces total bandwidth by ~40% for multi-page sessions. Cloudflare's CDN automatically caches responses with `s-maxage` at the edge.

Phase 2: Performance Optimization (Week 2)

2.1 — Static Generation (SSG) for the Main Page

Why: Currently, the page is a client-side SPA that renders nothing until JavaScript loads and fetches data. With SSG + ISR, the HTML arrives pre-rendered with product data already embedded.

Convert `page.tsx` to use ISR:

TypeScript

```typescript
// src/app/page.tsx — remove "use client" from the page level
// Instead, make it a server component that pre-renders with data

import { getProducts } from "@/lib/server-products";

export const revalidate = 300; // Regenerate every 5 minutes (ISR)

export default async function HomePage() {
  const products = await getProducts(); // Fetches at build time + every 5 min

  return (
    <HomeClient initialProducts={products} />
  );
}
```

Impact:

- First Contentful Paint: 3-5 seconds → <1 second
- Time to Interactive: 5-8 seconds → <2 seconds
- Google Core Web Vitals: Poor → Good (critical for SEO)
- Eliminates the "stuck at loading" problem entirely for first paint

---

2.2 — Implement Stale-While-Revalidate Pattern for Client Data

Why: Even with SSG, the client needs fresh data for stock levels and new products. The SWR pattern shows cached data instantly while refreshing in the background.

TypeScript

```typescript
// Replace the current polling approach with SWR
import useSWR from 'swr';

const fetcher = (url: string) => fetch(url).then(r => r.json());

export function useCatalog() {
  const { data, error, isLoading } = useSWR('/api/products', fetcher, {
    revalidateOnFocus: false,
    revalidateOnReconnect: true,
    refreshInterval: 300_000, // 5 minutes
    dedupingInterval: 60_000, // Dedup within 1 minute
    fallbackData: SEED_PRODUCTS, // Show something immediately
  });

  return { products: data || [], loading: isLoading, error };
}
```

Impact: Eliminates custom polling logic, reduces code complexity, and provides automatic deduplication (multiple components using the same hook share one request).

2.3 — Image Optimization Pipeline

Why: Current images are 450×600px / 22KB average. For a decor store, customers need to see product details. But higher-res images need optimization to avoid bandwidth explosion.

Strategy: Upload high-res originals to R2, serve optimized versions via Cloudflare Image Resizing:

TypeScript

```typescript
// Enable Cloudflare Image Resizing (requires Pro plan or $5/month add-on)
// URL format: /cdn-cgi/image/width=600,quality=80,format=auto/{image_url}

function getOptimizedImageUrl(src: string, width: number = 600): string {
  if (src.startsWith('https://images.soumdeco.com' )) {
    return `/cdn-cgi/image/width=${width},quality=80,format=auto/${src}`;
  }
  return src;
}
```

Alternatively (free), use R2 with pre-generated sizes:

Upload 3 sizes per image during the sync process:

- Thumbnail: 300×400px (for product grid) — ~15KB
- Medium: 600×800px (for product detail) — ~50KB
- Full: 1200×1600px (for zoom) — ~150KB

TypeScript

```
// Serve appropriate size based on context
const sizes = {
  grid: `${R2_BASE}/${id}-thumb.webp`,    // 15KB
  detail: `${R2_BASE}/${id}-medium.webp`, // 50KB
  zoom: `${R2_BASE}/${id}-full.webp`,     // 150KB
};
```

Impact: Better product presentation while keeping bandwidth under control. WebP format saves 30-50% vs JPEG.

2.4 — JavaScript Bundle Optimization

Why: With 64 dependencies (many unused), the JS bundle is likely 500KB+. For new visitors on Algerian mobile networks (3G/4G), this means 3-8 seconds of download time.

Actions:

1. **Remove unused dependencies** (saves ~1MB from node_modules, ~200KB from bundle):

   Bash

   ```
   npm uninstall next-auth next-intl sharp recharts react-syntax-highlighter \
     react-markdown z-ai-web-dev-sdk react-resizable-panels @tanstack/react-tab
     react-day-picker input-otp
   ```

2. **Remove Prisma** (saves ~5MB from node_modules):

   Bash

   ```
   npm uninstall prisma @prisma/client
   rm src/lib/db.ts
   ```

3. **Dynamic import heavy components:**

   TypeScript

```
// Admin panel — only loaded when admin logs in
const AdminPanel = dynamic(() => import('./admin-panel'), { ssr: false });

// Checkout modal — only loaded when user clicks checkout
const CheckoutModal = dynamic(() => import('./checkout-modal'), { ssr: false
```

4. **Analyze and split the bundle:**

```
Bash

ANALYZE=true next build
# Use @next/bundle-analyzer to identify large chunks
```

Target: Main bundle < 150KB gzipped (currently estimated 300-500KB).

Phase 3: Scalability & Reliability (Week 3)

3.1 — Order Submission: Dual-Write with Queue

Why: At 2% conversion (20,000 orders/month), order submission must be bulletproof. Apps Script can handle this volume, but network failures between Cloudflare and Google are possible.

Implement a Cloudflare Queue for guaranteed delivery:

```
TypeScript

// /api/order/route.ts
export async function POST(req: NextRequest) {
  const env = (req as any).env;
  const body = await req.json();

  // Validate server-side (phone, required fields)
  const validated = validateOrder(body);
  if (!validated.ok) return NextResponse.json(validated, { status: 400 });

  // Write to Queue (guaranteed delivery, retries automatically)
  await env.ORDER_QUEUE.send({
    ...validated.data,
    timestamp: Date.now(),
    orderId: generateOrderId(),
  });

  // Also try direct write (best-effort, for instant sheet update)
```

```
  fetch(APPS_SCRIPT_URL, { method: 'POST', body: JSON.stringify(validated.data)
    .catch(() => {}); // Queue guarantees delivery even if this fails

  return NextResponse.json({ ok: true, orderId: validated.data.orderId });
}
```

Configure in `wrangler.toml`:

Plain Text

```
[[queues.producers]]
binding = "ORDER_QUEUE"
queue = "soumdeco-orders"

[[queues.consumers]]
queue = "soumdeco-orders"
max_retries = 10
max_batch_size = 10
```

Impact: Zero lost orders, even during Apps Script downtime. Orders are guaranteed to reach the sheet within minutes.

Cost: Queues: $0.40/million messages = ~$0.01/month for 20K orders.

3.2 — Rate Limiting & Bot Protection

Why: At 1M visits/month, you'll attract bots, scrapers, and potential DDoS. Without protection, bots could consume your Apps Script quota or spam orders.

Implement via Cloudflare Workers middleware:

TypeScript

```
// middleware.ts
import { NextResponse } from 'next/server';

export function middleware(request: Request) {
  // Cloudflare provides cf-connecting-ip and cf-ray headers
  const ip = request.headers.get('cf-connecting-ip') || 'unknown';

  // Rate limit order submissions: 3 per IP per 5 minutes
  if (request.url.includes('/api/order') && request.method === 'POST') {
    // Use KV for simple rate limiting
    // Or use Cloudflare's built-in Rate Limiting rules (dashboard)
  }
```

```
  return NextResponse.next();
}
```

Better: Use Cloudflare's built-in tools (free tier):

- **Bot Fight Mode** (free): Blocks known bad bots
- **Rate Limiting Rules** (free tier: 1 rule): Limit `/api/order` to 5 req/5min per IP
- **Challenge pages** for suspicious traffic (free)

Impact: Prevents order spam, protects Apps Script quota, blocks credential stuffing on admin.

3.3 — Admin Authentication (Server-Side)

Why: The current client-side password check is trivially bypassable. At 1M visits, someone will find and exploit it.

Implement proper server-side auth using Cloudflare Workers KV sessions:

TypeScript

```typescript
// /api/admin/login/route.ts
export async function POST(req: NextRequest) {
  const env = (req as any).env;
  const { password } = await req.json();

  // Compare against env var (not hardcoded)
  if (password !== env.ADMIN_PASSWORD) {
    return NextResponse.json({ ok: false }, { status: 401 });
  }

  // Generate session token
  const token = crypto.randomUUID();
  await env.CATALOG_KV.put(`session:${token}`, "admin", { expirationTtl: 86400 }

  // Set httpOnly cookie
  return NextResponse.json({ ok: true }, {
    headers: {
      'Set-Cookie': `admin_session=${token}; HttpOnly; Secure; SameSite=Strict;
    },
  } );
}
```

Impact: Admin operations are protected by server-validated sessions. Password is never exposed in client bundle.

3.4 — Apps Script Authentication

Why: The exposed Apps Script URL allows anyone to read products or submit fake orders.

Add a shared secret token to Apps Script:

JavaScript

```
// In Apps Script (apps-script.gs)
function doGet(e) {
  const token = e.parameter.token || '';
  if (token !== PropertiesService.getScriptProperties().getProperty('AUTH_TOKEN
    return ContentService.createTextOutput(JSON.stringify({ error: 'unauthorize
      .setMimeType(ContentService.MimeType.JSON);
  }
  // ... existing logic
}
```

Pass the token from the edge API (server-side only, never exposed to browser):

TypeScript

```
// In /api/products/route.ts
const res = await fetch(`${sheetUrl}?action=products&token=${env.APPS_SCRIPT_TO
```

Impact: Apps Script can only be called from your Cloudflare Workers (which have the secret). Direct browser calls are blocked.

Phase 4: SEO & Growth (Week 4)

4.1 — Add Product Pages for SEO

Why: With 1M visits/month target, you need organic search traffic. Currently, Google can't index individual products because everything is a client-side modal on one URL.

Add dynamic product routes:

TypeScript

```
// src/app/product/[slug]/page.tsx
import { getProducts, getProductBySlug } from "@/lib/server-products";

export const revalidate = 300; // ISR: regenerate every 5 min

export async function generateStaticParams() {
```

```
  const products = await getProducts();
  return products.map(p => ({ slug: p.id }));
}

export async function generateMetadata({ params }) {
  const product = await getProductBySlug(params.slug);
  return {
    title: `${product.name} | SOUM DECO`,
    description: product.description,
    openGraph: {
      images: [product.image],
    },
  };
}

export default async function ProductPage({ params }) {
  const product = await getProductBySlug(params.slug);
  return <ProductDetail product={product} />;
}
```

Impact: Each product gets its own URL, meta tags, and structured data. Google indexes 80+ product pages. Enables social media sharing of individual products.

4.2 — Structured Data (JSON-LD) for Rich Snippets

TypeScript

```
// In product page
const jsonLd = {
  "@context": "https://schema.org",
  "@type": "Product",
  name: product.name,
  image: product.images,
  description: product.description,
  offers: {
    "@type": "Offer",
    price: product.price,
    priceCurrency: "DZD",
    availability: product.inStock
      ? "https://schema.org/InStock"
      : "https://schema.org/OutOfStock",
  },
};
```

Impact: Google shows rich product snippets (price, availability, image) in search results — significantly higher click-through rates.

4.3 — Sitemap & Robots.txt

TypeScript

```typescript
// src/app/sitemap.ts
export default async function sitemap() {
  const products = await getProducts();
  return [
    { url: 'https://soumdeco.com', lastModified: new Date( ), priority: 1.0 },
    ...products.map(p => ({
      url: `https://soumdeco.com/product/${p.id}`,
      lastModified: new Date( ),
      priority: 0.8,
    })),
  ];
}
```

Cost Summary

Service	Current (Free)	Required for 1M Visits	Monthly Cost
Cloudflare Pages	Free	Free (unlimited bandwidth)	**$0**
Cloudflare Workers (Paid)	Free (100K/day)	Paid (10M included)	**$5**
Cloudflare KV (with Workers Paid)	100K reads/day	10M reads/month included	**$0** (included)
Cloudflare R2	Not used	500MB storage, 4M reads	**$0** (free tier)
Cloudflare Queues	Not used	~20K messages/month	**$0.01**
Google Apps Script	Free	Free (under 20K calls/day with KV cache)	**$0**
Cloudinary	Free (25GB)	Not needed (migrated to R2)	**$0**
Domain (if not already owned)	—	.com domain	~$10/year

TOTAL	**$0**		**~$5/month**

The entire infrastructure to handle 1 million visits/month costs approximately $5/month — the Cloudflare Workers paid plan. Everything else fits within free tiers thanks to edge caching.

Performance Targets

Metric	Current	After Optimization	Target
Time to First Byte (TTFB)	2-5s (Apps Script)	<50ms (edge)	<100ms
First Contentful Paint	3-5s	<1s	<1.5s
Largest Contentful Paint	5-8s	<2s	<2.5s
Time to Interactive	5-10s	<2.5s	<3s
JS Bundle (gzipped)	~400KB est.	<150KB	<200KB
API response time	2-5s	<50ms	<100ms
Image load time	1-3s	<500ms	<1s
Order submission	3-8s	<1s	<2s
Uptime	~95% (Apps Script dependent)	99.9%+	99.9%

Implementation Priority & Timeline

Week 1 (Critical — Must Do)

1. Migrate to `@opennextjs/cloudflare` (restores API routes)
2. Implement KV caching for products/stock (eliminates rate limit risk)
3. Migrate images to R2 (eliminates Cloudinary bandwidth limit)
4. Upgrade to Workers Paid plan ($5/month)

Week 2 (Performance — Should Do)

1. Implement SSG/ISR for homepage (instant first paint)
2. Remove unused dependencies (smaller bundle)
3. Dynamic import admin panel + checkout (code splitting)
4. Add proper HTTP cache headers

Week 3 (Security & Reliability — Should Do)

1. Server-side admin authentication
2. Apps Script token authentication
3. Order queue (guaranteed delivery)
4. Rate limiting + bot protection

Week 4 (Growth — Nice to Have)

1. Product pages with SEO
2. Structured data (JSON-LD)
3. Sitemap generation
4. Image optimization pipeline (multiple sizes)

Risk Mitigation

Risk	Mitigation
Apps Script goes down	KV cache serves stale data (5-min old). Visitors see products. Orders queue for retry.
Traffic spike (viral moment)	Cloudflare edge absorbs 100% of static/cached traffic. Only cache misses hit origin.
R2 outage	Fall back to Cloudinary URLs (keep as backup in image manifest).
KV cache corruption	Auto-expire after 5 minutes. Worst case: one 5-min window of stale data.
Order submission failure	Cloudflare Queue retries up to 10 times over 24 hours. localStorage backup on client.

DDoS attack	Cloudflare's free DDoS protection + rate limiting rules.

Quick-Start Implementation Checklist

For the developer implementing this, here's the exact sequence:

Plain Text

```
☐ 1. npm uninstall @cloudflare/next-on-pages
☐ 2. npm install -D @opennextjs/cloudflare
☐ 3. Update wrangler.toml (see Section 1.1)
☐ 4. Update package.json build script: "build:cloudflare": "npx @opennextjs/clo
☐ 5. Test locally: npx wrangler dev
☐ 6. Verify /api/products returns data (not 500)
☐ 7. Add KV caching to /api/products and /api/stock (see Section 1.2)
☐ 8. Update use-catalog.ts to fetch from /api/products (not Apps Script directl
☐ 9. Create R2 bucket: npx wrangler r2 bucket create soumdeco-images
☐ 10. Upload images: wrangler r2 object put (see Section 1.3)
☐ 11. Update image URLs to use R2 public domain
☐ 12. Remove unused deps (see Section 2.4)
☐ 13. Upgrade Cloudflare to Workers Paid ($5/month)
☐ 14. Deploy and verify: git push → wait 2 min → test all flows
☐ 15. Monitor: Check Cloudflare Analytics for error rates
```

Conclusion

The path from "breaks at 150K visits" to "handles 1M+ visits" requires surprisingly few changes — primarily because Cloudflare's infrastructure is designed for exactly this scale. The core insight is simple: **never let a visitor's browser talk directly to Google Apps Script.** By interposing a 5-minute KV cache at the edge, you reduce origin load by 99.6% and serve every visitor in <50ms regardless of their location.

The total cost increase is $5/month (Cloudflare Workers Paid plan). Everything else — R2 storage, KV reads, Pages bandwidth, Queue messages — fits within free tiers at 1M visits/month.

The site's existing self-healing mechanisms (localStorage cache, seed data fallback, retry queues) remain valuable as defense-in-depth, but they shift from being the primary reliability strategy to being a last-resort safety net behind a properly cached edge architecture.